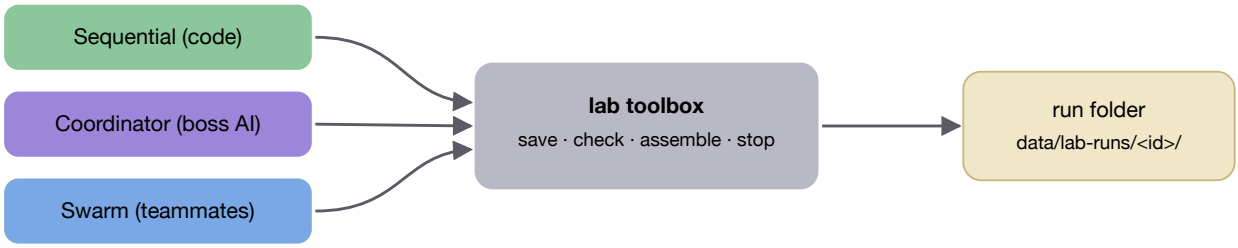
How the three teams are built

The implementation, in plain words, with the real code. Companion to "Three ways to run the same team". 29 October 2026.

The big idea: one toolbox, three ways to use it

The three teams don't each get their own rules. They all use **one toolbox**, a small command line called `lab`, and every rule lives inside it. So when a team does better, it's because of how it's wired, not because it got easier rules.



The toolbox commands

Command	What it does
`lab new`	Starts a run, and reads the website if you gave one
`lab save`	The only way to hand in work. Checks it's valid before keeping it
`lab check`	Runs the Ruler (the code checks) and says **pass**, **redo** or **stop**
`lab assemble`	The Glue. Only works once every check has passed
`lab stop`	Gives up on a run. A stopped run is never a success
`lab report` · `compare`	Time, tokens, fixes needed, and every run side by side

The rules live in code, not in a prompt

An AI can ignore an instruction, but it can't get past code. Here's the part of `lab save` that enforces "two tries to fix it":

apps/backend/src/lab/commands.ts

```
const attempt = run.attempts[key] + 1;
if (attempt - 1 > MAX_RETRIES_PER_OWNER) {        // MAX_RETRIES_PER_OWNER = 2
  const reason = `${owner} out of retries`;
  await finishRun(env, cleared, "stopped", reason);  // the run ends "stopped"
  throw new LabError(reason);                        // and the agent is told why
}
```

And there's one more rule: three invalid saves in a row (bad JSON, the wrong shape, a plan that renames things) end the run as "failed".

1. Sequential: the order is a loop in code

Code calls the Director, runs the Ruler, and either moves on, asks for a fix, or stops. The AI never decides what happens next.

apps/backend/src/lab/sequential.ts

```
for (;;) {
  // the AI writes a plan; code saves it through the toolbox
  await produce(deps, runId, "plan", "director", askTheDirector);
  // code checks it: pass, redo or stop
  const check = await labCheck(env, runId, "plan", "lab");
  if (check.route.kind === "pass") break;            // next: the Cinematographer
  if (check.route.kind === "stop") return final();   // out of tries
  print(`[route] redo director: ${check.route.checkIds.join(", ")}`);
}
```

You watch it as one line per step: `[director] plan 1`, `[checks] plan: 4 of 5 passed`, `[route] redo director: length_fits`.

2. Coordinator: the helpers are files

Each helper is a Claude Code agent file. Code writes these files from the same instructions the sequential team uses, so the helpers are identical in all three teams.

`.claude/agents/lab-director.md (generated)`

```
---
name: lab-director
tools: Bash, Read, SendMessage
model: claude-sonnet-5-5
effort: high
---
## If you were called as a subagent (the coordinator)
Do your part, then return two lines: what you saved and what the check printed.
The lead decides what happens next.
```

The boss is your Claude Code session, on Opus 5.5 at high effort. `/direct` gives it the goal and the rules, but not the order. Choosing the order is its job.

3. Swarm: the helpers pass the ball themselves

The same three files, started as teammates. Each one checks its own work and messages the next. The lead only passes your answers along.

`.claude/skills/direct/SKILL.md (the swarm part)`

```
You run on Opus 5.5 at high effort, but you are only the doorman:
you pass messages between the owner and the team.
You never plan, route, order or fix anyone's work.
```

`.claude/agents/lab-cinematographer.md (the swarm part)`

```
- When the shot checks pass, run `npm run -s lab -- assemble <runId>`
  and tell the `lead` "done: <runId>".
- You may ask `director` about a shot before you shoot it.
```

Same instructions everywhere

The new prompt versions are made by changing exact lines of the old ones. If an old line ever moves, the code stops with an error, so a prompt can't quietly drift. The Director's only change: with no photo, it may now describe how the product looks.

`apps/backend/src/agents/director/prompt-v2.ts`

```
let lines = buildDirectorSystemPrompt().split("\n");   // v1, untouched
// the product line: "describe only its kind" becomes "say what a camera sees"
lines = replaceOnce(lines, (l) => l.startsWith(V1_PRODUCT_LINE_START),
                    () => V2_PRODUCT_LINE);
// the location line loses "painted as a picture"
lines = replaceOnce(lines, (l) => l.includes(V1_PICTURE_SENTENCE),
                    (l) => l.replace(V1_PICTURE_SENTENCE, ""));
```

A test checks that the agent files on disk still match these prompts exactly.

How to test it yourself

Team	Start Claude Code with	Then type
Sequential	`claude` (needs `ANTHROPIC_API_KEY` in `.env`)	`/direct`
Coordinator	`claude --model claude-opus-5-5 --effort high`	`/direct`
Swarm	in iTerm2 or tmux: `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 claude --model claude-opus-5-5 --effort high --teammate-mode tmux`	`/direct`

After a run: `npm run -s lab -- compare` shows every run side by side. Each run's files are in `data/lab-runs/<id>/`: `shotlist.md`, `prompt.txt`, `trace.jsonl` (who did what) and `metrics.json`.

What is proven, and what isn't yet

- **Proven:** every rule in the toolbox, the sequential loop (pass, fix-then-pass, stop, fail), the scraper and the generated agent files, all with tests that use fake AI answers. The whole project suite passes.
- **Not proven yet:** how the real models behave. No paid AI call was made while building this. Your nine test runs (3 ideas × 3 teams) are the experiment, and the report for your teacher gets written from them.
- **Never:** no pictures and no video. Nothing here can call a generation API.